DeepAR (Auto Regression)

Probabilistic Forecasting with Autoregressive Recurrent Networks [1]

IEOR 4578: Forecasting: *A Real-World Application*

Syed Waseem Haider, Ph.D.

Dr.Waseem.Haider@gmail.com

April 15, 2026

[1] David Salinas, Valentin Flunkert, Jan Gasthaus, Tim Januschowski, DeepAR: Probabilistic Forecasting with Autoregressive Recurrent Networks, International Journal of Forecasting, Volume 36, Issue 3, 2020, Pages 1181-1191, ISSN 0169-2070

Introduction

- Reference Texts include:
  - Deep Learning by Ian Goodfellow, Yoshua Bengio, and Aaron Courville. Online version is available by authors here
  - Neural Networks and Deep Learning: A Textbook, 2019, by Charu C. Aggarwal.
- The DeepAR paper is available here.
- Code for the DeepAR implementation on the Electricity data set is available here.

Distinct Advantages over the Classical Methods

- As the model learns seasonal behaviors and dependencies on given covariates across time series, minimal manual intervention in providing covariates is needed in order to capture complex, group dependent behavior.
- DeepAR makes probabilistic forecasts in the form of Monte Carlo samples that can be used to compute consistent quantile estimates for all sub-ranges in the prediction horizon.
- By learning from similar items, our method is able to provide forecasts for items that have little or no history available, a case where traditional singleitem forecasting methods fail.
- The approach does not assume Gaussian noise, but can incorporate a wide range of likelihood functions, allowing the user to choose one that is appropriate for the statistical properties of the data.
- The second and fourth points are instrumental in the production of accurate, calibrated forecast distributions that are learned from the historical behavior of all of the time series jointly.

Link with RNN, LSTM, and Encoder-Decoder

- Recall that RNN models a dynamic system driven by an external input x_t.
- The hidden state, h_t, is modeled by $h_t = f(h_{t-1}, x_t; \theta)$.
- Furthermore, the problems of vanishing and exploding gradients are solved using LSTM with the cell structures.
- Next, the use of Encode-Decoder with RNN to map sequence to sequence. Instead of mapping a fixed length input to a fixed length output using RNN, Encoder and Decoder RNNs allow to map a variable length input to a variable length output sequence.
- The output of an Encoder RNN is a fixed length Context, C. C is the abstract representation h_t of the input sequence $\{x_t\}$, and it is mapped through a Decoder RNN to the output sequence $\{y_t\}$.
- The two RNNs jointly maximizes the average $log\ P(\{y_t\}|\{x_t\})$ of the pairs of input-output sequences over the training set.

DeepAR

- Recall that the AR(p) process is $y_t = c + \varepsilon_t + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p}$ is the *pth*-order difference equation.
- Analogous to the AR process, $\{z_t\}$ is the output sequence, and $\{x_t\}$ is the sequence of a covariate, where$\{z_t\}$ is randomly distributed.
- The Encoder-Decoder RNNs are used to model the distribution of $\{\hat{z}_t\}$ using the likelihood function $\hat{z}_t \sim \ell\left(\, . \mid \theta(h_t, \Theta)\right)$.
- Hence redfining the hidden state $h_t = f\left(h_{t-1}, z_{t-1}, x_t; \theta\right) = f\left(h_{t-1}, z_{t-1}, x_t; \Theta\right)$.
- The updated model for the i-th electricity consumer is $P\left(z_{i,t_0:T} \mid z_{i,1:t_0-1}, x_{i,1:T}\right)$.

Deeap AR

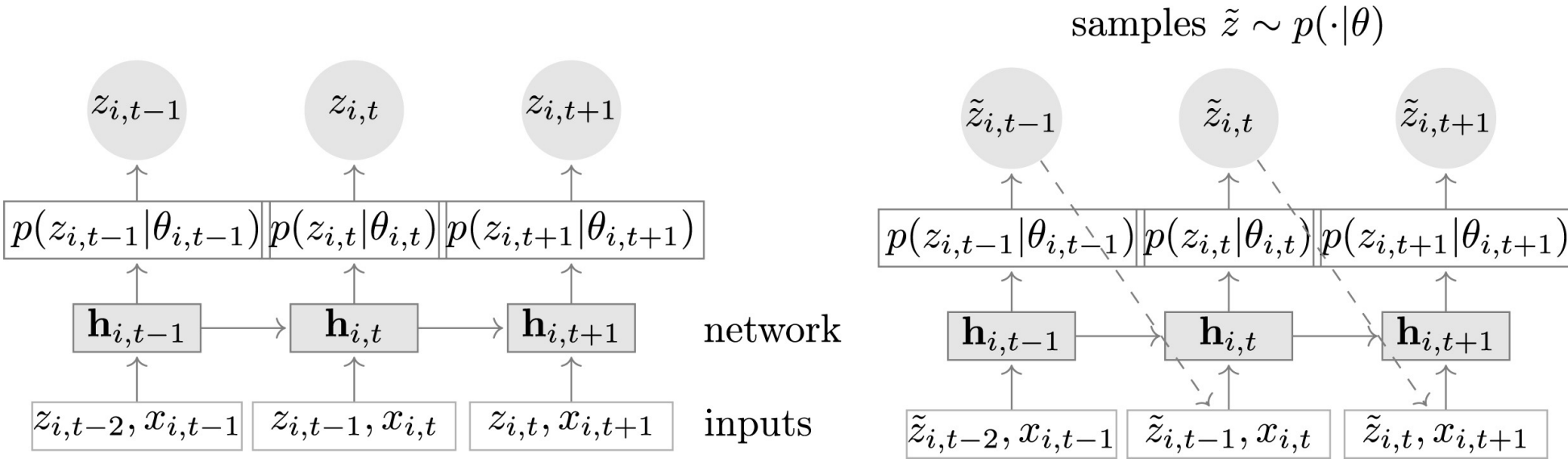

Figure: Model Summary of DeepAR.